

ClaimVault Security Audit

OpenZeppelin Security - January 15, 2026

Summary

Type: DeFi

Timeline: From 2025-10-30 → To 2025-10-31

Languages: Solidity

Findings

Total issues: 10 (10 resolved)

Critical: 0 (0 resolved) · High: 0 (0 resolved) · Medium: 1 (1 resolved) · Low: 1 (1 resolved)

Notes & Additional Information

8 notes raised (8 resolved)

Scope

OpenZeppelin conducted a security audit of the [ZeroBase-Pro/ZKFi](#) repository at commit [3413ed5](#).

In scope was the following file:

This site uses cookies to offer you a better user experience. Information about the cookies we use can be found in our Privacy Policy under "Cookie Policy". You can reject all non-essential cookies by choosing "Decline" or set your own preference under "Cookies settings".

[Cookies settings](#)

Accept

Decline

System Overview

The ZKFi protocol by ZEROBASE allows for the staking of stablecoins that back a zero-knowledge-powered proof network. `ClaimVault` is the component that governs the controlled distribution of ZBT rewards to eligible participants. Claims are authorized off-chain by a designated signer and gated on-chain by per-epoch global and per-user limits, with the owner retaining authority to pause the contract, adjust epoch parameters, rotate the signer, and perform emergency withdrawals. Ensuring the correctness of signature handling and the integrity of the epoch-capped accounting is critical because the contract holds user rewards in custody.

The system's distinguishing feature is its epoch-based throttling of claimable amounts, combining both a global and per-user rate limit in tandem with off-chain signature authorization. This layered mechanism allows the project operator to safely distribute a finite pool of tokens over time without relying on continuous on-chain state updates or complex vesting schedules.

Security Model and Trust Assumptions

The `ClaimVault` contract assumes that the off-chain signer operates honestly and securely, as this entity effectively defines user eligibility and allocation limits. Any compromise of the signer's private key could result in unauthorized claims being accepted by the contract. Similarly, users must trust that the owner configures epoch parameters appropriately and maintains sufficient token liquidity for claim fulfillment.

The system's security relies on the correctness of the ECDSA signature scheme provided by the OpenZeppelin ECDSA library, as well as the integrity of Ethereum's chain ID mechanism for cross-chain replay protection. External dependencies such as the ERC-20 token implementation (ZBT) are assumed to conform to the standard interface and to behave as expected under the `SafeERC20` interface.

Operationally, the owner can pause/unpause distribution, change the rate-limiting parameters at any time, replace the signer, and remove all escrowed tokens through emergency withdrawal. Therefore, the owner is trusted to operate the system honestly, maintain the signer's private key security, and configure epoch limits that align with the intended reward cadence.

Lastly, it is assumed that the project operator deploys a dedicated instance of `ClaimVault` for each signer on each chain. This deployment pattern prevents signature reuse or replay across parallel vaults that might otherwise share the same signer credentials. By binding each contract instance to a unique signer-chain pair, the protocol ensures that signatures cannot be validly replayed across multiple vaults operating on the same network.

Privileged Roles

The `ClaimVault` contract defines a single privileged role, the Owner, derived from the OpenZeppelin `Ownable` base contract. The owner possesses full administrative control, including updating the authorized signer, adjusting epoch configuration parameters, pausing or resuming claim operations, and performing emergency withdrawals of tokens. This ownership model implies unilateral authority: any governance or multisig enforcement must occur externally, as no timelocks or multi-party checks are implemented in the contract.

The Signer is not an on-chain role but a critical off-chain actor, responsible for issuing valid claim signatures recognized by the contract. While not endowed with transaction authority, the signer defines the logical access control over claims and thus forms a core trust assumption in the system's security model.

Medium Severity

Possible Cross-Contract Signature Replay Attack

The `calculateClaimZBTHash` function of the `ClaimVault` contract generates a digest for signature verification without including the contract's address (`address(this)`) in the hash calculation. This omission presents a potential security vulnerability in scenarios where the same signer key is utilized across multiple `ClaimVaults` with identical nonce-handling mechanisms. In such cases, a legitimately signed message for one contract could be replayed on another, leading to unauthorized claims or other unintended actions.

Currently, the deployment plan for the `ClaimVault` contract envisages a single instance. However, considering the possibility of future deployments or extensions of the contract framework, it is prudent to preemptively address this replay attack vector. Incorporating the contract's address into the signature hash effectively binds the signature to the specific contract instance, significantly mitigating the risk of cross-contract replay attacks.

Consider modifying the `calculateClaimZBTHash` function to include `address(this)` in the hash calculation. This adjustment ensures that signatures are explicitly tied to the contract instance, preventing their reuse across different contracts.

Update: Resolved in [pull request #2](#) at commit [710f42f](#).

Low Severity

Missing Docstrings

All functions in the `ClaimVault` contract are missing docstrings. This hinders reviewers' understanding of the code's intention, which is fundamental to correctly assessing not only security but also correctness. In addition, docstrings improve readability and ease maintenance. Thus, they should explicitly explain the purpose or intention of the functions, the scenarios under which they can fail, the roles allowed to call them, the values returned, and the events emitted.

Consider thoroughly documenting all functions (and their parameters) that are part of the contracts' public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Resolved in [pull request #3](#) at commit [cebf380](#).

Notes & Additional Information

Multiple Optimizable State Reads

Within the `Claim` function of the `ClaimVault` contract, there are multiple storage reads of the `epochDuration` variable. This leads to inefficient use of gas due to repeated SLOAD operations. By caching this variable's value in memory at the beginning of the function, it is possible to reduce the gas cost associated with these operations.

Consider reducing SLOAD operations that consume unnecessary amounts of gas by caching the values in a memory variable.

Update: Resolved in [pull request #4](#) at commit [c129d79](#).

Function Visibility Overly Permissive

The `_checkSignature` function has an `internal` visibility, which may be more permissive than necessary for its intended use within the contract. Adjusting function visibility to the strictest level necessary is a best practice in smart contract development, as it not only clarifies the intended use and access level of functions, but can also lead to gas savings when the functions are compiled and deployed.

Consider changing the visibility of the `_checkSignature` function from `internal` to `private` to better convey its intended use and to potentially gain some additional gas savings.

Update: Resolved in [pull request #5](#) at commit [8175854](#).

Inconsistent Order Within `ClaimVault`

The `ClaimVault` contract exhibits an inconsistency with the recommended practices outlined in the Solidity Style Guide, particularly in the ordering of its functions. The contract places several `external` functions after `internal` ones, diverging from the suggested order that aims to enhance code readability and maintainability.

To improve the project's overall legibility, consider standardizing ordering throughout the codebase as recommended by the [Solidity Style Guide \(Order of Functions\)](#).

Update: Resolved in [pull request #6](#) at commit [37c3163](#).

Missing Security Contact

The `ClaimVault` contract does not have a security contact. Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Consider adding a NatSpec comment containing a security contact above the contract definition. Using the `@custom:security-contact` convention is recommended as it has been adopted by the

OpenZeppelin Wizard and the [ethereum-lists](#).

Update: Resolved in [pull request #7](#) at commit [6c2715b](#).

Use of Magic Numbers

The `ClaimVault` contract initializes the `epochDuration`, `globalCapPerEpoch`, and `userCapPerEpoch` variables with specific values that are not accompanied by explanatory comments or documentation. These values, often referred to as "magic numbers," can lead to confusion and make the codebase less accessible to future developers or auditors who may not understand the rationale behind these initial settings. The presence of the `setEpochConfig` function, which allows an admin to modify these variables, further emphasizes the need for clarity regarding the initial values of these variables.

To improve the readability of the codebase, consider not initializing the aforementioned state variables to magic numbers and properly setting these state variables during the construction of the `ClaimVault` contract.

Update: Resolved in [pull request #8](#) at commit [6763596](#).

Redundant Balance Check in `Claim`

The `Claim` function of the `ClaimVault` contract includes a balance check to ensure that the `ClaimVault` address has sufficient balance before proceeding with the claim operation. However, this check is rendered unnecessary by the subsequent use of the `safeTransfer` function. The `safeTransfer` function inherently checks that the sender (in this case, the `ClaimVault` contract) has enough tokens to cover the transfer amount. If the balance is insufficient, `safeTransfer` will revert the transaction, making the initial balance check superfluous.

Consider removing the explicit balance check before the `safeTransfer` call to optimize the `Claim` function for better efficiency and simplicity.

Update: Resolved in [pull request #9](#) at commit [7b31bcd](#).

Use Custom Errors

Since Solidity version 0.8.4, custom errors provide a cleaner and more cost-efficient way to explain to users why an operation failed.

Multiple instances of `revert` and/or `require` messages were identified within `ClaimVault.sol`:

- The `require(claimAmount != 0, "Zero ZBT number")` statement
- The `require(user == msg.sender, "Invalid sender")` statement
- The `require(expiry > block.timestamp, "Signature expired")` statement
- The `require(_checkSignature(claimDigestHash, signature), "Invalid signature")` statement

- The `require(globalUsed + claimAmount <= globalCapPerEpoch, "Global cap exceeded")` statement
- The `require(userUsed + claimAmount <= userCapPerEpoch, "User cap exceeded")` statement
- The `require(ZBT.balanceOf(address(this)) >= claimAmount, "Insufficent Balance")` statement
- The `require(_token != address(0), "Token must not be zero")` statement
- The `require(_receiver != address(0), "Receiver must not be zero")` statement
- The `require(_newSigner != address(0), "Signer must not be zero")` statement
- The `require(_epochDuration > 0, "epochDuration can not be zero")` statement
- The `require(_globalCapPerEpoch > 0, "globalCapPerEpoch must greater than zero")` statement
- The `require(_userCapPerEpoch > 0 && _userCapPerEpoch <= _globalCapPerEpoch, "_userCapPerEpoch must greater than zero and less than _globalCapPerEpoch")` statement.

For conciseness and gas savings, consider replacing `require` and `revert` messages with custom errors.

Update: Resolved in [pull request #10](#) at commit [9c8e152](#).

Function Names Should Use mixedCase

As per the [Solidity Style Guide](#), function names should use mixedCase.

The `Claim` function of the `ClaimVault` contract uses the CapWords style instead of mixedCase.

To better adhere to the Solidity Style Guide, consider using mixedCase for function names.

Update: Resolved in [pull request #11](#) at commit [9d38683](#).

Conclusion

The `ClaimVault` contract introduces a controlled, signature-based token claim mechanism designed to enable time-bound and rate-limited distributions of an ERC-20 asset. The system leverages off-chain

authorization and on-chain enforcement of epoch-based claim caps, providing a secure and flexible means for projects to manage token releases over time. Overall, the codebase demonstrates a clear structure and strong adherence to best practices. A single medium-severity vulnerability and several issues of lesser severity were identified during the review. In general, the codebase was found to be well-written, self-contained, and easy to understand.

The ZEROBASE team was responsive and provided great clarity throughout the audit process. Their forthcomingness to provide context and clarification contributed significantly to the effectiveness of this review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood

of being exploited or may cause unwanted side effects in the system. These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.

Ready to secure your code?

[Request an Audit→](#)

Smart Contracts

Contracts Libraries

Contracts Wizard ↗

Contracts MCP ↗

Upgrades Plugin ↗

Open Source Tools

Relayer

Monitor

UI Builder ↗

Company

About Us

Why OpenZeppelin

News

Customer Stories

Careers

Brand Kit

Trust ↗

Terms & Policies

[Safe Utils ↗](#)

[Role Manager ↗](#)

[The Open Source Stack](#)

Blockchains

[Arbitrum](#)

[Base](#)

[Ethereum](#)

[Linea](#)

[Midnight](#)

[OP Mainnet](#)

[Polkadot](#)

[Starknet](#)

[Stellar](#)

[Zama](#)

[ZKsync](#)

[Terms of Service](#)

[Privacy Policy](#)

[Trademark Guidelines](#)

[AGPL License](#)

Services

[Smart Contract Security Audit](#)

[Blockchain Infrastructure Audit](#)

[ZKP Audit](#)

[Security Services](#)

Solutions

[Ecosystem Stack](#)

[Financial Institutions Stack](#)

Resources

[Research](#)

[Documentation ↗](#)

[Stats](#)

[Forum ↗](#)

[Ethernaut CTF ↗](#)

